```
#include "utils.hpp"
#include "part1.hpp"
#include "part2.hpp"
#include <Arduino.h>
#include <Servo.h>


Servo servoLeft;
Servo servoRight;

void setup() {
  Serial.begin(9600);
  servoLeft.attach(servoLeftPin);
  servoRight.attach(servoRightPin);

  pinMode(LeftirReceiverPin, INPUT);
  pinMode(FrontirReceiverPin, INPUT);
  pinMode(RightirReceiverPin, INPUT);

  pinMode(LeftredLedPin, OUTPUT);
  pinMode(FrontredLedPin, OUTPUT);
  pinMode(RightredLedPin, OUTPUT);

  stop();
  delay(1000);

  task4();

}
```

```
void loop() {
}
```

```
#include "utils.hpp"
#include "part1.hpp"

static void badLeftParallel() {
  turnRightXDeg(20); // clockwise
  forwardMs(500);     // straight
  turnLeftXDeg(20);   // anti-clockwise
  backwardMs(200);    // backward
}

static void badRightParallel() {
  turnLeftXDeg(20);   // anti-clockwise
  forwardMs(500);     // straight
  turnRightXDeg(20);  // clockwise
  backwardMs(200);    // backward
}

static void badLeft30Deg() {
  turnRightXDeg(30);  // undo the 30 deg
  badLeftParallel();  // now it's the parallel case
}

static void badRight30Deg() {
  turnLeftXDeg(30);
```

```
    badRightParallel();
}


void startingPos() {
    readSensors();

    bool nearLeft = dL <= NEAR;
    bool nearRight = dR <= NEAR;
    bool detectLeft = dL < FAR_DIST;
    bool detectRight = dR < FAR_DIST;

    if (nearLeft && !nearRight) {
        if (detectRight) badLeftParallel();
        else badLeft30Deg();
    }
    else if (nearRight && !nearLeft) {
        if (detectLeft) badRightParallel();
        else badRight30Deg();
    }
}

// drive striaght until there is a front wall, adjusting if off center
void straightUntilWall() {
    straight();
    while (true) {
        readSensors();
        displayLEDwall();

        if (dF <= FRONT_TURN_MAX) break;
```

```
      if (dF == FAR_DIST && dL <= NEAR && dR > NEAR) badLeftParallel();
      else if (dF == FAR_DIST && dR <= NEAR && dL > NEAR) badRightParallel();
      else steer();
    }
    stop();
    readSensors();
}


// turn to whichever side is open
void turnToOpenSide() {
    if (dL > dR) turnLeftXDeg(90);
    else       turnRightXDeg(90);
}

// check if dead end
bool atDeadEnd() {
    return dL < OPEN && dR < OPEN;
}

void task1() {
    startingPos();
    straightUntilWall();
}

void task2() {
    task1();
    turnToOpenSide();
    straightUntilWall();
}
```

```
void task3() {
  task1();
  while (!atDeadEnd()) {
    turnToOpenSide();
    straightUntilWall();
  }
}

//task 3 twice
void task4() {
  task3();

  if (dL <= dR) {turnLeftXDeg(180);}
  else {turnRightXDeg(180);}
  task3();
}
```

```
#include "utils.hpp"

int dF = FAR_DIST, dL = FAR_DIST, dR = FAR_DIST;

int irDetect(int irLedPin, int irReceiverPin, long frequency)
{
 tone(irLedPin, frequency);            // Turn on the IR LED square wave
 delay(1);                             // Wait 1 ms
 int ir = digitalRead(irReceiverPin);  // IR receiver -> ir variable
 noTone(irLedPin);                     // Turn off the IR LED
 delay(1);                             // Down time before recheck
 return ir;                            // Return 0 detect, 1 no detect
```

```
}

// one reading of all three sensors into dF, dL, dR (same calls as main_controller's loop)
void readSensors() {
  dF = irDistanceCm(FrontirLedPin, FrontirReceiverPin);
  dL = irDistanceCm(LeftirLedPin,  LeftirReceiverPin);
  dR = irDistanceCm(RightirLedPin, RightirReceiverPin);
}

void displayLEDwall() {
  digitalWrite(LeftredLedPin,  dL < OPEN);
  digitalWrite(FrontredLedPin, dF <= FRONT_TURN_MAX);
  digitalWrite(RightredLedPin, dR < OPEN);
}

void straight() {
  servoLeft.writeMicroseconds(LEFT_FWD_US);
  servoRight.writeMicroseconds(RIGHT_FWD_US);
}

void backward() {
  servoLeft.writeMicroseconds(RIGHT_FWD_US);
  servoRight.writeMicroseconds(LEFT_FWD_US);
}

void stop() {
  servoLeft.writeMicroseconds(STOP_US);
  servoRight.writeMicroseconds(STOP_US);
}

void veerLeft() {
```

```
  servoLeft.writeMicroseconds(LEFT_FWD_US - STEER);
  servoRight.writeMicroseconds(RIGHT_FWD_US);
}

void veerRight() {
  servoLeft.writeMicroseconds(LEFT_FWD_US);
  servoRight.writeMicroseconds(RIGHT_FWD_US + STEER);
}

// to keep centered while driving
void steer() {
  bool leftWall  = dL < OPEN;
  bool rightWall = dR < OPEN;
  int err = 0;

  if (leftWall && rightWall) err = dR - dL;
  else if (leftWall)        err = WALL_CM - dL;
  else if (rightWall)       err = dR - WALL_CM;

  if (err >= SIDE_DIFFERENCE) veerRight(); // closer to left wall, so turn slightly right
  else if (err <= -SIDE_DIFFERENCE) veerLeft(); // closer to right wall, so turn slightly left
  else straight(); // else go straight
}

void forwardMs(int ms)  { straight(); delay(ms); stop(); }
void backwardMs(int ms) { backward(); delay(ms); stop(); }

void turnRightXDeg(int deg) { rotateRight(); delay(spinMs(deg, true));  stop(); }
void turnLeftXDeg(int deg)  { rotateLeft();  delay(spinMs(deg, false)); stop(); }
```

```
void rotateLeft() {
 servoLeft.writeMicroseconds(1455);
 servoRight.writeMicroseconds(1455);
}

void rotateRight() {
 servoLeft.writeMicroseconds(1525);
 servoRight.writeMicroseconds(1525);
}



int irDistanceCm(int irLedPin, int irReceiverPin)
{
 long maxF = 0;

 // Scan from high to low so we get closest dist first
 for (long f = 49000; f >= 37000; f -= 500)
 {
  if (irDetect(irLedPin, irReceiverPin, f) == 0)
  {
   maxF = f;
   break;
  }
 }

 if (maxF == 0) return FAR_DIST; // no detection

 // boundary conditions on our table
 if (irReceiverPin == FrontirReceiverPin) {
  if (maxF <= CAL_FREQ_FRONT[0])    return CAL_DIST_FRONT[0];
  if (maxF >= CAL_FREQ_FRONT[N - 1]) return CAL_DIST_FRONT[N - 1];
```

```
 } else if (irReceiverPin == LeftirReceiverPin) {
  if (maxF <= CAL_FREQ_LEFT[0])     return CAL_DIST_LEFT[0];
  if (maxF >= CAL_FREQ_LEFT[N - 1])  return CAL_DIST_LEFT[N - 1];
 } else if (irReceiverPin == RightirReceiverPin) {
  if (maxF <= CAL_FREQ_RIGHT[0])     return CAL_DIST_RIGHT[0];
  if (maxF >= CAL_FREQ_RIGHT[N - 1]) return CAL_DIST_RIGHT[N - 1];
 }

 // choose dist between the calibration points roughly linear estimate
 for (int i = 0; i < N - 1; i++) {
  if (irReceiverPin == FrontirReceiverPin) {
   if (maxF <= CAL_FREQ_FRONT[i + 1])
    return map(maxF, CAL_FREQ_FRONT[i], CAL_FREQ_FRONT[i + 1], CAL_DIST_FRONT[i],
CAL_DIST_FRONT[i + 1]);
  } else if (irReceiverPin == LeftirReceiverPin) {
   if (maxF <= CAL_FREQ_LEFT[i + 1])
    return map(maxF, CAL_FREQ_LEFT[i], CAL_FREQ_LEFT[i + 1], CAL_DIST_LEFT[i],
CAL_DIST_LEFT[i + 1]);
  } else if (irReceiverPin == RightirReceiverPin) {
   if (maxF <= CAL_FREQ_RIGHT[i + 1])
    return map(maxF, CAL_FREQ_RIGHT[i], CAL_FREQ_RIGHT[i + 1], CAL_DIST_RIGHT[i],
CAL_DIST_RIGHT[i + 1]);
  }
 }

 return FAR_DIST;
}
```

```
#ifndef UTILS_HPP
#define UTILS_HPP

#include <Arduino.h>
#include <Servo.h>

extern Servo servoLeft;
extern Servo servoRight;

// pins
const int FrontirLedPin=6, FrontirReceiverPin=7;
const int FrontredLedPin = A1;
const int LeftirLedPin=10, LeftirReceiverPin=11;
const int LeftredLedPin = A2;
const int RightirLedPin=2, RightirReceiverPin=3;
const int RightredLedPin = A0;

const int servoLeftPin = 13;
const int servoRightPin = 12;

// IR calibration table
const int N = 5;
const long CAL_FREQ_FRONT[N] = {38500, 40000, 41000, 42000, 45750};
const int  CAL_DIST_FRONT[N] = {10, 8, 6, 5, 3};

const long CAL_FREQ_LEFT[N] = {38500, 39000, 40750, 41500, 45250};
const int  CAL_DIST_LEFT[N] = {10, 8, 6, 5, 3};

const long CAL_FREQ_RIGHT[N] = {37750, 39250, 40500, 41500, 44500};
const int  CAL_DIST_RIGHT[N] = {10, 8, 6, 5, 3};
```

```
const int FAR_DIST = 99;

// for classification
const int NEAR = 3; // shortest dist
const int OPEN = 10;  // bigger than 10 is open wall
const int SIDE_DIFFERENCE = 2; // difference that counts as "off centre"
const int FRONT_CLEAR = 10; // move forward in open path
const int FRONT_TURN_MIN = 3;  // wall ahead at turning distance
const int FRONT_TURN_MAX = 8;  // between 3 and 8 cm

// speeds  or turn rates
const int  STOP_US = 1490;
const int  STANDARD_LEFT_FORWARD = 1600, STANDARD_RIGHT_FORWARD = 1375;

const int WALL_CM = 6;
const int STEER = 40;

// latest readings, filled in by readSensors()
extern int dF, dL, dR;

// sensors
int  irDetect(int irLedPin, int irReceiverPin, long frequency);
int  irDistanceCm(int irLedPin, int irReceiverPin);
void readSensors();
void printSensors(const char* tag);
void displayLEDwall();

// wheels - these just set the servos and return (no delay)
void straight();
void backward();
```

```
void stop();
void veerLeft();
void veerRight();
void rotateLeft();
void rotateRight();
void steer();
long spinMs(int deg, bool right);

// wheels - these block (delay) until the move is done
void forwardMs(int ms);
void backwardMs(int ms);
void turnRightXDeg(int deg);
void turnLeftXDeg(int deg);

#endif
```